Karlsruher Institut für Technologie

PUNCH Young Academy Training
PUNCH4NFDI Compute and Storage Resources

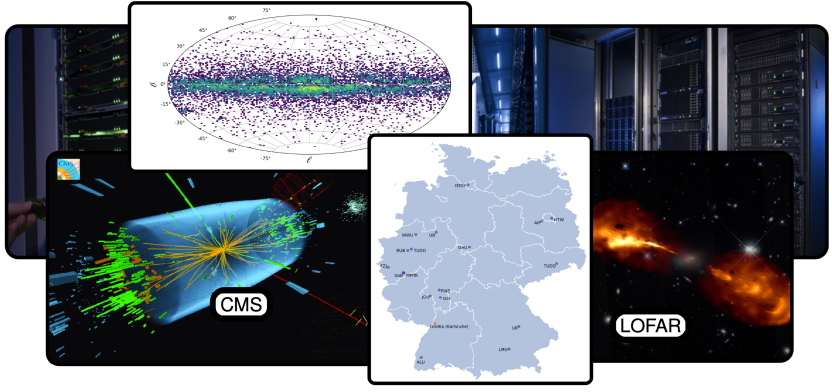
PUNCH4NFDI Annual Meeting 2024 - Bonn

Benoit Roland (KIT), Manuel Giffels (KIT)



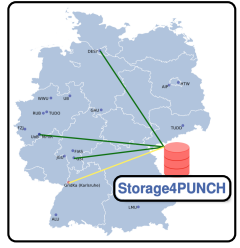
Overview

- Infrastructure overview
- Access compute resources
- Access storage resources
- Analysis workflow
- Hands on session

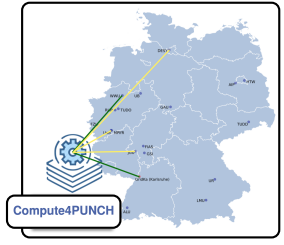


Infrastructure pillars

- PUNCH4NFDI has to cope with increasing amount of data and compute need
- Integrate available **storage** resources into a federated storage platform
  → **Storage4PUNCH** (Bonn, DESY, GSI, KIT)
- Federate available **compute** resources
  → **Compute4PUNCH** (Göttingen, LMU, LRZ Cloud, KIT, WWU)
- Provide **transparent access** to these resources
  → **Single entry point**
- **Secure** access to these resources
  → **Token-based Authentication**

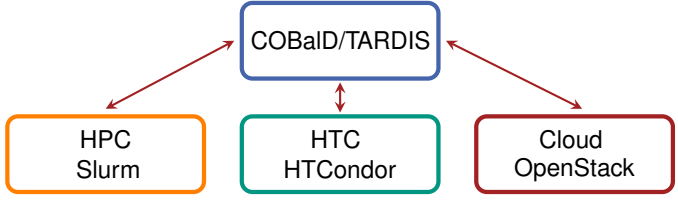






Compute4PUNCH key ingredients

- Heterogenous compute resources with different schedulers, architectures, operating systems
- Aggregate resources in a single **Overlay Batch System** based on **HTCondor**
- Provision resources to the OBS using **COBalD/TARDIS meta-scheduler**
  → Single federated pool with dynamic extension on a user-demand basis
- **TARDIS** integrates available resources into OBS
- **COBalD** does balancing, job to resource matching, ensures effective usage



Resource provisioning and payload assignment

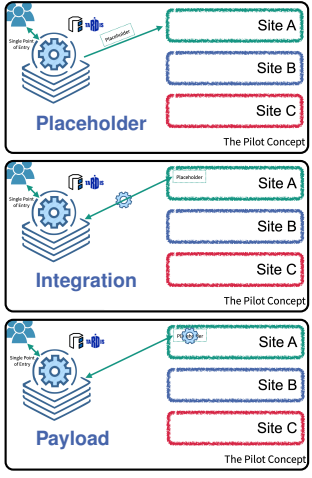
- Placeholder job - a drone - is submitted to the site
- Drone allocates and integrates resource into the OBS and provides a specific environment via a container
- Payload pulled from the OBS and assigned to the resource

User interacts with the OBS through a single entry point:

- Login node - provided
- JupyterHub - expected soon
- OIDC token-based authentication

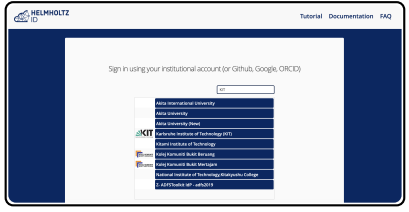


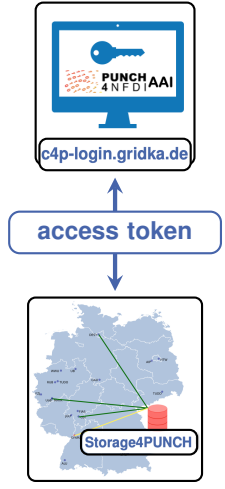




Storage4PUNCH

- Endpoints at DESY and KIT using dCache
- Endpoints at Bonn and GSI using XRootD
- WebDav/XRootD as transfer protocol
- **Token-based access using Helmholtz AAI**





- Register once to Helmholtz AAI provider
- Once registered - access token generated with oidc-agent
- Storage4PUNCH accessed from login node

Access token

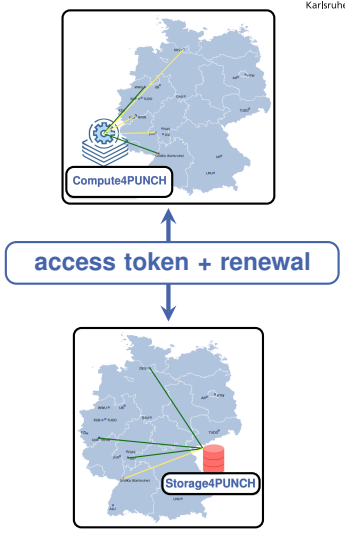
- Helmholtz AAI access tokens are mandatory to access Storage4PUNCH from Compute4PUNCH worker nodes

→ Have to be **inserted** in the HTCondor job

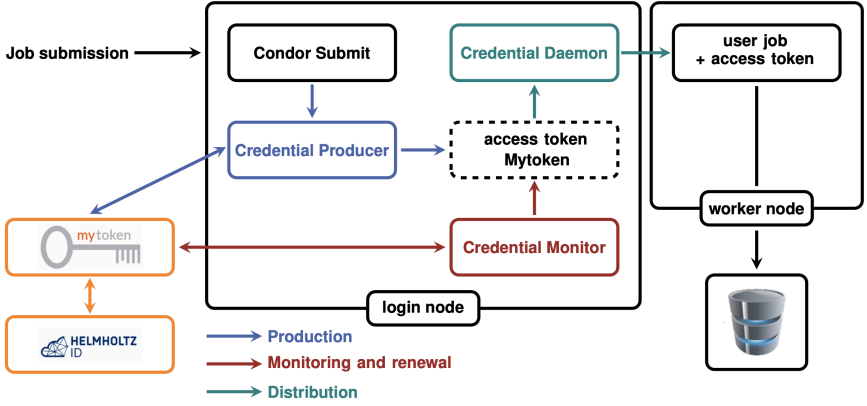
- Access tokens have a limited life time of about one hour

→ Have to be **renewed** for jobs exceeding their life time





Access token management



- Long-living credential artifact known as **Mytoken** not exposed to the world
- Used to provision new access tokens **without resorting to the user**

Local setup to access login node

Compute resources accessed through single entry point → **C4P login node** `c4p-login.gridka.de`

Connection to C4P login node → via access tokens issued by **Helmholtz AAI provider**

- **Create Helmholtz AAI user account**
- **Install oidc-agent** on your local host → command line tool **to handle OIDC access tokens**
- **Configure oidc-agent** for Helmholtz AAI provider on your local host: `oidc-gen --pub punch-aai`
- **Install SSH-client-wrapper mccli** on your local host → **SSH connection with OIDC access tokens**



SSH world	OIDC world
ssh-agent	oidc-agent
SSH keys	OIDC tokens
ssh client	mccli ssh client



Connection to login node

- Start oidc-agent and define OIDC_SOCK environment variable: `eval 'oidc-agent'`
- Add punch-aai configuration to oidc-agent: `oidc-add punch-aai`
- Activate python virtual environment for mccli: `source mccli_venv/bin/activate`
- Connect to C4P login node: `mccli ssh c4p-login.gridka.de`

```
Last login: Tue Nov 19 15:30:45 2024 from 2a02:8071:b688:1bc0:e435:68c4:fb8d:fdd1
This host is managed by Puppet.

Config Management Information
 Satellite Host: https://satellite.scc.kit.edu/hosts/c4p-login.gridka.de
 Hostgroup: PUNCH/login
 Puppet Environment: production
 Satellite Owner: Benoit Roland <benoit.roland@kit.edu>
 i-doit Host: Virtual machines are not registered in i-doit


This machine is a Compute4PUNCH login node. Have fun!

[benoit_roland@c4p-login ~]$
```

Define agent socket path

- To avoid having to enter your password several times in a given session

  → **define socket path for the agent**

```
oidc_sock="$HOME/.cache/oidc-agent/oidc-agent.sock"
export OIDC_SOCK=$oidc_sock
if [ !  -S $oidc_sock ]; then
  oidc-agent -socket-path=$oidc_sock -console > /dev/null 2>&1 &
fi
oidc-add punch-aai
```

- Can be implemented as a function in your profile configuration

→ **Will be addressed in Hands on session**

OIDC agent command lines

- **Generate new account configuration:** `oidc-gen --pub punch-aai`
- **Delete existing account configuration:** `oidc-gen --delete punch-aai`
- **Load configuration to agent:** `oidc-add punch-aai`
- **List available configurations:** `oidc-add --list`
- **List loaded configurations:** `oidc-add --load`
- **Decrypt and print configuration:** `oidc-add --print punch-aai`
- **Generate OIDC access token:** `oidc-token punch-aai`

→ **Will be addressed in Hands on session**

Access storage resources

Endpoints

- Endpoints at DESY and KIT using dCache

  DESY: `https://dcache-desy-webdav.desy.de:2880//pnfs/desy.de/punch/`

  KIT: `https://dcache-kit.gridka.de:2880/pnfs/gridka.de/punch4nfdi/`

- Endpoints at Bonn and GSI using XRootD

  Bonn: `https://xrootd.physik.uni-bonn.de:1094/cephfs/grid/helmholtz`

  GSI: `https://lxpunch1.gsi.de:1094/`

File management tool

- **DaviX client** to transfer and manage files using HTTP/WebDav protocols
- Authentication based on **Helmholtz AAI access tokens** and **X509 certificates**
- Access token provided in the "Authorization" header using the "Bearer" HTTP authentication scheme

List content of DESY storage element

Define three elements

- **Storage element URL:**
  `export DESY_STORAGE=https://dcache-desy-webdav.desy.de:2880//pnfs/desy.de/punch/`
- **X509 certificate directory:**
  `export X509_CERT_DIR=/cvmfs/grid.cern.ch/etc/grid-security/certificates/`
- **Access token:** `export BEARER_TOKEN=$(oidc-token punch-aai)`

```
davix-ls --capath $X509_CERT_DIR -l -H "Authorization:  Bearer $BEARER_TOKEN" $DESY_STORAGE
```

```
[benoit_roland@c4p-login ~]$ eval `oidc-agent`
Agent pid 3932554
[benoit_roland@c4p-login ~]$ oidc-add punch-aai
Enter decryption password for account config 'punch-aai':
success
[benoit_roland@c4p-login ~]$ export BEARER_TOKEN=$(oidc-token punch-aai)
[benoit_roland@c4p-login ~]$ export X509_CERT_DIR=/cvmfs/grid.cern.ch/etc/grid-security/certificates/
[benoit_roland@c4p-login ~]$ export DESY_STORAGE=https://dcache-desy-webdav.desy.de:2880//pnfs/desy.de/punch/
[benoit_roland@c4p-login ~]$ davix-ls --capath $X509_CERT_DIR -l -H "Authorization: Bearer ${BEARER_TOKEN}" $DESY_STORAGE
drwxrwxrwx 0     0          2024-10-14 15:19:47 alexander
drwxrwxrwx 0     0          2024-06-25 13:17:13 atlas_opendata_2020
drwxrwxrwx 0     0          2024-06-26 16:06:18 baida
drwxrwxrwx 0     0          2024-05-21 17:25:28 benoit_roland
drwxrwxrwx 0     0          2023-08-14 22:03:24 c4p-cern-open-data-demo
drwxrwxrwx 0     0          2024-06-17 15:28:07 HEP-OpenData
drwxrwxrwx 0     0          2024-03-14 12:54:56 manuel_giffels
drwxrwxrwx 0     0          2024-03-06 20:12:41 prateek
-rwxrwxrwx 0     51942      2023-08-18 10:26:09 PUNCH_logo.png
drwxrwxrwx 0     0          2024-11-20 09:29:23 rucio
drwxrwxrwx 0     0          2024-04-23 10:47:19 tutorials
drwxrwxrwx 0     0          2024-04-02 16:54:14 user
[benoit_roland@c4p-login ~]$
```

DaviX tool command lines

Download file

```
davix-get --capath $X509_CERT_DIR -H "Authorization:  Bearer $BEARER_TOKEN"
$DESY_STORAGE/PUNCH_logo.png PUNCH_logo.png
```

Upload file

```
davix-put --capath $X509_CERT_DIR PUNCH_logo.png -H "Authorization:  Bearer $BEARER_TOKEN"
$DESY_STORAGE/PUNCH_logo.png
```

Create directory

```
davix-mkdir --capath $X509_CERT_DIR -H "Authorization:  Bearer $BEARER_TOKEN"
$DESY_STORAGE/user/user_name
```

Delete file or directory

```
davix-rm --capath $X509_CERT_DIR -H "Authorization:  Bearer $BEARER_TOKEN"
$DESY_STORAGE/user/user_name/result.root
```

All access tokens are mapped to a single user! That means you can delete files created by other PUNCH users!

Scripts to manage storage resources

Scripts are available on the login node to manage storage resources

```
[benoit_roland@c4p-login ~]$ s4p-list-storage.sh
Specify the storage element: gridka, bonn, desy, gsi
Specify the directory to be listed (no specification will list the entire storage element)
[benoit_roland@c4p-login ~]$
[benoit_roland@c4p-login ~]$ s4p-get-storage.sh
Specify the storage element: gridka, bonn, desy, gsi
Specify the file to be retrieved
[benoit_roland@c4p-login ~]$
[benoit_roland@c4p-login ~]$ s4p-mkdir-storage.sh
Specify the storage element: gridka, bonn, desy, gsi
Specify the directory to be created
[benoit_roland@c4p-login ~]$
[benoit_roland@c4p-login ~]$ s4p-remove-storage.sh
Specify the storage element: gridka, bonn, desy, gsi
Specify the directory or the file to be removed
[benoit_roland@c4p-login ~]$
[benoit_roland@c4p-login ~]$ s4p-list-storage.sh gridka user/benoit_roland

Listing the content of the directory https://dcache-kit.gridka.de:2880/pnfs/gridka.de/punch4nfdi/user/benoit_roland

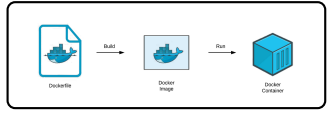
basic-test
test-directory

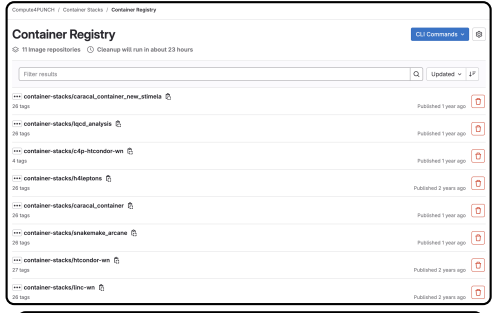
[benoit_roland@c4p-login ~]$
```

→ **Will be addressed in Hands on session**

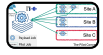
Typical analysis workflow

- Containerise analysis software
- Make it available in PUNCH4NFDI container registry
- Distribution via the CVMFS file system





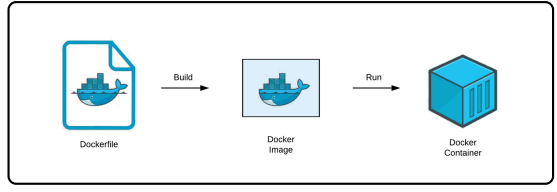
- Job submission from login node 

- HTCondor worker node started 

- Job executed with container retrieved from CVMFS

Analysis containerisation

- Compute4PUNCH workflow based on **Docker container**
- **Containerise:** Package elements of an application to be able to run it **on any compute environment**
  Virtual compute environment: own operating system, software and dependencies
  Only dependence on physical environment via kernel
- **Image:** Template defining what will be inside container
  Similar to a class
- **Container:** Executable instance of an image
  Similar to an object
  One single image - several containers



- **Docker file:** Set of instructions defining the image → **Only** ingredient **you** need to set up

→ **Building image** and **running container** are done within **Compute4PUNCH**

Docker file content

- **Set up framework on physical resource:** Run commands on the command line in a shell
- **Docker file** will contain all these commands preceded by **instructions to assemble image**
- Each instruction will create **a layer in the Docker image** - onion-like structure
- **FROM:** first instruction - used to define your base image

  `FROM rockylinux:8` → Operating System

  `FROM cmsopendata/cmssw_5_3_32-slc6_amd64_gcc472:latest` → CMS compute environment
- **RUN:** execute specified command to create a new layer in the image

  `RUN dnf -y install python3.11` → install python libraries

  `RUN git clone https://github.com/lofar-astron/LOFARBeam.git` → clone git repository

  `RUN mkdir -p LOFARBeam/build` → create directory structure

  `RUN chmod +x execute.sh` → set executable permission

Docker file instructions

- **COPY:** copy local file or directory to the image file system

```
COPY execute.sh /srv/execute.sh
```

- **ADD:** copy remote file via URL

```
ADD https://github.com/krallin/tini/releases/download/v0.19.0/tini /tini
```

- **ENV:** set environment variables

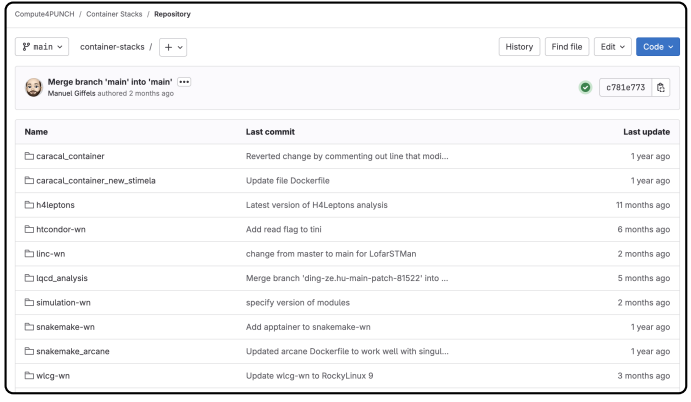
```
ENV TINI_VERSION v0.19.0
ADD https://github.com/krallin/tini/releases/download/$TINI_VERSION/tini /tini
```

- **WORKDIR:** set working directory for next instructions

```
RUN git clone https://github.com/rstofi/arcane_suite.git
WORKDIR /arcane_suite
RUN ...
```

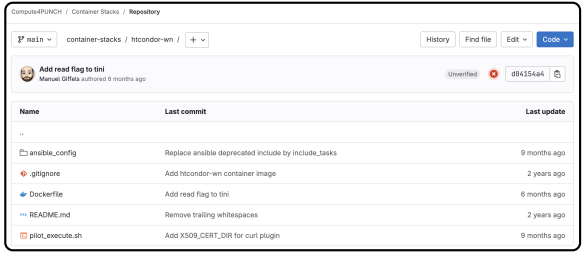
Compute4PUNCH Container Stacks - overview

- At this stage: Docker file - and any elements to be present inside the image - **on your local machine**
- Next step: Make that available in the Compute4PUNCH **Container Stacks Repository** in GitLab



Compute4PUNCH Container Stacks - clone and push

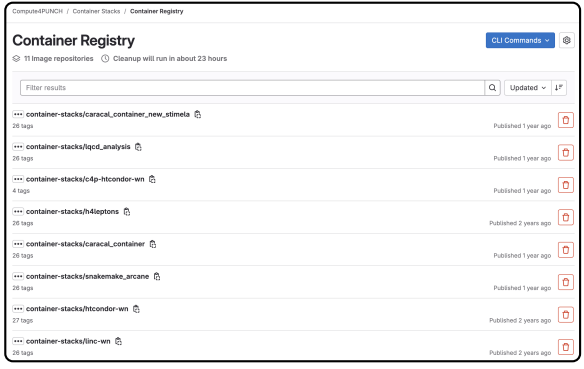
- **Clone** the Container Stacks Repository and create feature branch
- **Add** there your Docker file - and any elements to be present inside the image
- **Push** your change and create a merge request → **That's done!**



- HTCondor worker node: Dockerfile and two elements required to build the image

Compute4PUNCH Container Registry

- Your Docker file is now available in the Container Stacks Repository
- CI/CD pipeline will **build the image** and **upload it** to the Compute4PUNCH **Container Registry**



Distribution via CVMFS

- Your image is now available in the Container Registry
- An unpacked version is automatically made available in the CVMFS file system
- When running your analysis on HTCondor, your container will be retrieved from CVMFS

```
[benoit_roland@c4p-login ~]$ ls /cvmfs/unpacked.cern.ch/gitlab-p4n.aip.de\:5005/compute4punch/container-stacks/
caracal_container:latest              htcondor-wn:latest     simulation-wn:latest     wlcg-wn:latest
caracal_container_new_stimela:latest  linc-wn:latest        snakemake_arcane:latest
h4leptons:latest                      lqcd_analysis:latest  snakemake-wn:latest
[benoit_roland@c4p-login ~]$
```

- It's now time to connect to the login node to submit your job!

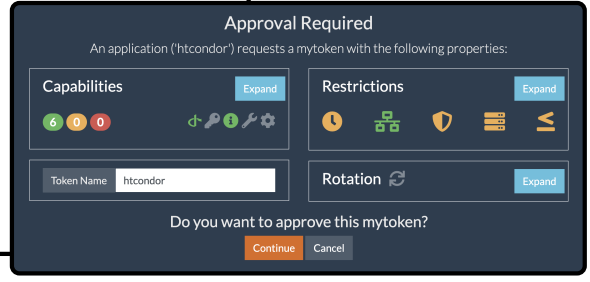
Job configuration

```
executable = analysis.sh
use_oauth_services = helmholtz  → Specify Helmholtz AAI prodiver
should_transfer_files = YES
output_destination =
helmholtz+https://dcache-desy-webdav.desy.de:2880//pnfs/desy.de/punch/analysis
output = logs/$(machine).$(cluster).$(process).out
error = logs/$(machine).$(cluster).$(process).err
log = logs/$(machine).cluster.log
request_cpus = 1
request_memory = 256
+SINGULARITY_JOB_CONTAINER = "analysis:latest"
queue 1
```

Job submission

```
condor_submit analysis.jdl
Submitting job(s)
Hello benoit_roland!  You are going to submit your HTCondor jobs.
No credential has been found!
Please visit the following url in order to generate your credential:
https://mytoken.data.kit.edu/c/XWSdsgNC
Starting polling and waiting for your approval...
Your credential has been successfully created!
Its remaining life time is 6 d 23 h 58 m 46 s.
1 job(s) submitted to cluster 1142
```

→ **Visit specified URL and approve Mytoken**



Hands on

- OIDC agent
- Storage management
- Run dimuon spectrum analysis
- Job monitoring

Hands on repository

References

- Max Fischer, Eileen Kuehn, Manuel Giffels, Matthias Schnepf, Stefan Kroboth, Thorsten M., Oliver Freyermuth. **MatterMiners/cobald (0.14.0) 2023.**
- Manuel Giffels, Max Fischer, Alexander Haas, Stefan Kroboth, Matthias Schnepf, Eileen Kuehn, P. Schuhmacher, Rene Caspart, Florian von Cube, Peter Wienemann. **MatterMiners/tardis (0.8.2) 2024.**
- **CVMFS** - CERN Virtual Machine File System.
- **Mytoken** - Web service to provide OIDC access tokens to long-running compute jobs.
- **C4P-HTCondor** - HTCondor with OIDC access tokens monitoring.